

Implementation: The Witness

Author: David Eslava

Version: 0.1

Date: 25 September 2026

1. Introduction

In this document I explain what I built for The Witness and how I built it. It is the fourth phase of the project, after the Analysis, the Advice and the Design.

I want to be honest from the start. This is my first game and I started with Godot on 15 September. So the prototype is small. At the moment you can walk around the jazz club with the player character, the walls stop you, and the rooms have lights and shadows. The guards, the game over screen and The Hunter are not built yet. I explain this in section 7.

2. Tools I used

- **Godot 4** as the game engine. I explain why I chose it in the Advice document.
- **GDScript** for the code. It looks a lot like Python, and I already know some basic Python.
- **Mixamo** for the 3D mannequin and the animations (walk, run, idle, crouch).
- **Blender** to change the style of the mannequin and to render the frames for the sprites.
- **Obsidian with the Git plugin** to save my notes and my project to GitHub.
- **YouTube tutorials**, mainly the Brackeys beginner tutorial, to learn the basics of Godot.

3. How I worked, step by step

15 September. I started the Brackeys tutorial "How to make a Video Game - Godot Beginner Tutorial". I made a small platformer called `first-game`. I learned that everything in Godot is made of nodes, and that scenes are groups of nodes that you can reuse. This project is also in my repository, because some parts (like the patrolling enemy) are useful for later.

18 September. I worked on the script of the character so it plays the correct animation for every action. My coach Faruk told me to look at `match` instead of many `if` lines.

21 September. My first version had around 50 lines of `if` and `elif`, one for every direction. I changed this to one `match` block that picks the direction. The code became

much shorter and easier to read.

22 September. I created the real project, `the-witness`. I added all the sprite frames for the player: idle, walk, run and crouch, in 8 directions.

23 September. I made the first map scene with a `TileMapLayer` and a `tileset`.

24 September. I tested the movement a lot. In the morning I removed the acceleration because I thought it felt slow, but without it the character stopped too hard, so I put it back. I also moved the camera inside the player so it follows him. In the evening I changed the size of the sprite, because the sprites were 1024 pixels and much too big for the world.

25 September. I built the full map of the jazz club with walls and lights. I used my map sketch from the Design document as a guide, so the rooms are in the same place as in the plan.

4. The player character

The player is a `CharacterBody2D` node. This is the node Godot uses for characters that move with code and hit walls. Inside it there is an `AnimatedSprite2D` for the animations and a `CollisionShape2D` (a circle) for the body.

All the code is in `player_character.gd`. It runs in `_physics_process`, so it runs many times every second. This is what it does:

1. It reads the keys with `Input.get_vector("left", "right", "up", "down")`. This gives one direction, also for diagonals.
2. It checks if the player holds Ctrl to crouch or Shift to run. Crouching makes the speed half. Running makes it five times faster. Right now there is no stamina, so the player can run forever.
3. It moves the player. I use `move_toward` with an acceleration and a friction value, so the player speeds up and slows down smoothly and does not start or stop in one frame. I also use `normalized()` so the player does not go faster on diagonals.
4. It calls `move_and_slide()`. This moves the body and makes it slide along walls instead of stopping dead.
5. It picks the animation. A `match` block turns the direction into a name like `Up_Right` or `Down_Left`. Then it joins the state and the direction, for example `Running_Up_Right`, and plays that animation.
6. When the player stops, it remembers the last direction and plays the idle animation (or crouch idle) for that direction. This way the character keeps looking the same way.

The speed, acceleration and friction use `@export`. This means I can change them in the Godot editor without touching the code. This was very useful when I was testing how

the movement feels.

5. The sprites

At first I wanted to draw a realistic top-down character myself. This was too hard for me and it took too much time. So I changed my plan. I took a mannequin with animations from Mixamo. In Blender I changed the look to black and white with nodes, to fit the noir style. Then I put the camera above the character and rendered the frames. I used around one frame of every four, so the files stay small and the animation still looks smooth.

I think this change was a good lesson. When something is too hard, I can look for another way to get the same result.

6. The map and the lights

The map follows the maze layout from the Design document. The player starts in the jazz hall, with the bar, the tables and the stage. From there you go through the kitchen, the toilets, the service corridor, the storage, the office, the construction zone and the lobby.

This is how the map is made:

- My map sketch is the background. I put it under everything, so I could draw the walls exactly on top of the lines.
- The walls are dark shapes with collision. The player cannot walk through them.
- The tables are circles with collision, and the bar counters are rectangles with collision. So the furniture also blocks the player.

For the lights, the whole scene is dark and every room has its own lamps. The light comes out in cone shapes. The walls and the tables block the light and make shadows. This gives the noir look I wanted, where most of the screen is dark and only some parts are lit. It also prepares the idea of the game: later, the player will hide in the dark parts.

You can see the result in the screenshots and in the gameplay video in my portfolio.

7. What is not built yet

Some parts of the Design are not in the game yet:

- **The guards.** There is no guard yet. The Design explains how it will work: a patrol route, a vision cone, pathfinding with `NavigationRegion2D`, and three states (Patrol, Chase, Caught). I also have a patrolling enemy from the Brackeys tutorial (`slime.gd`) that I can use as a starting point.

- **The game over screen and the exit.** Because there is no guard, the player cannot be caught yet. There is also no end screen when you reach the back door.
- **The stamina bar** for running.
- **The Hunter** with Q-learning. This was a Could Have, and my coach Faruk agreed that I do not build it in this challenge. The full design is in the Design document.

So the game at this moment is a level you can explore, not a game you can win or lose. The next step is clear: build one guard in the jazz hall (G1) and connect it to a game over screen.

8. Version control

My whole project is in my GitHub repository: <https://github.com/Eslavic/Fontys-University>. The game is in the folder `4 Realisation/Godot/the-witness`.

I save to GitHub with the Obsidian Git plugin. It makes a commit automatically every few minutes, with the message "vault backup" and the date. This is good because I never lose work. One time I deleted some images by mistake, and I could get them back from an older commit. But it also has a problem. The messages do not say what I changed, so the history is hard to read for other people. Next time I will make my own commits with clear messages, like "Add crouch animation" or "Fix diagonal speed".

9. What I learned

- How nodes and scenes work in Godot, and which node to use for a moving character.
- How to write movement code with acceleration, friction and diagonal movement.
- That a `match` block is often cleaner than many `if` lines.
- How to make sprites from 3D animations in Blender.
- How 2D lights and shadows work in Godot.
- That I should start the main mechanic (the guard) earlier. I spent a lot of time on the look of the game and on documents, and now the part that makes it a stealth game is still missing.